Smart View update

How it works, what changed, and how to carry on

A knowledge-transfer guide for the development team, in plain English. It explains the flow from the moment a person opens a source to the moment a file preview opens, lists every change made in this work, and says clearly what was tested and what was not.

Read this first	
Short version	Smart View is now a configurable table for any source (structure, data source, API). A source stays a plain table until someone sets it up. Views are kept on the server. Columns can be resized and moved. API data can hold files.
Where the code is	connectum/chat-app/features/smartView/ (screen), connectum-backend/src/forms/sd_smartview.erl and sd_svviews.erl (server)
Where the longer docs are	connectum/docs/SMART_VIEW.md, CONNECTUM_API.md, PERMISSIONS.md, UI_DESIGN.md
Status	Built and tested locally. Not committed, not pushed, no pull request (owner's instruction).

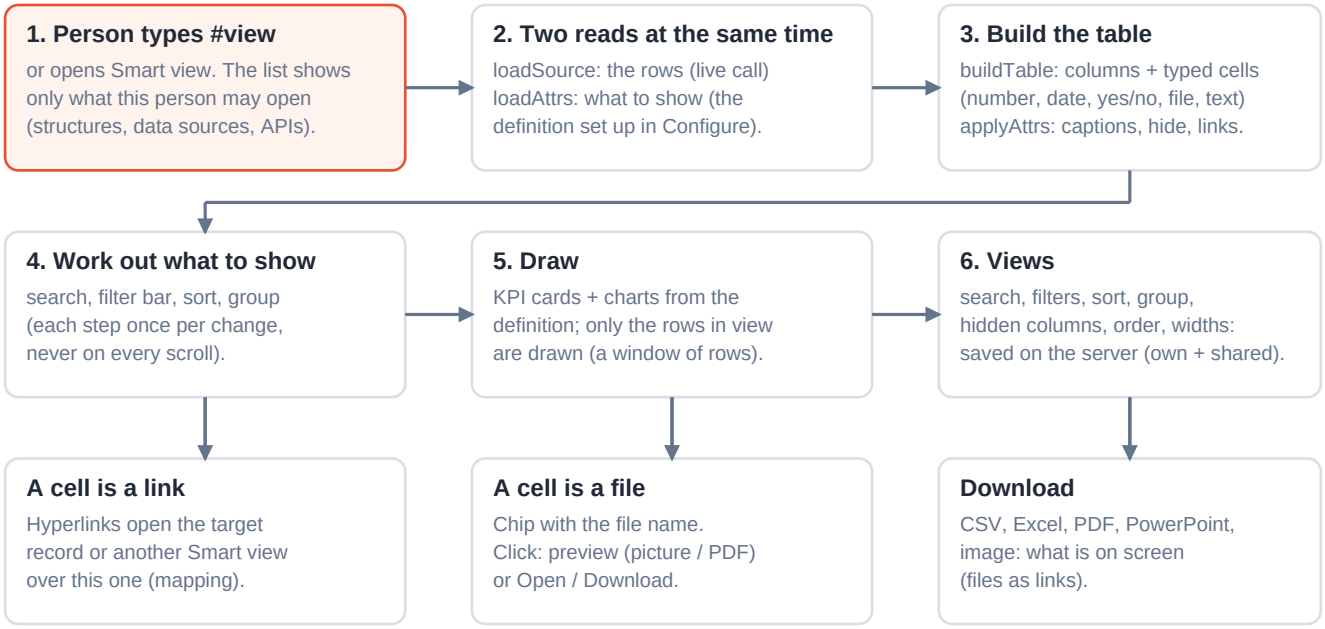
1. What changed, in simple words

Before this work, Smart View showed a source as a table and worked out some cards and charts by itself. Now an administrator decides what each source shows, and people can keep their own screen layouts. These are the changes, in the order they were built:

#	Change	In simple words	Who sees it
1	Smart view attributes (Configure)	An admin chooses, for each source, which columns show, what they are called, which are links, which KPI cards and which charts appear. Until that is done the source is a plain table with no cards or charts. Nothing is hard-coded any more.	Admins set it up; everyone sees the result
2	Hyperlinks	A cell can open another structure record or another Smart View. The admin maps a column of this table to a field of the target.	Everyone
3	Filter panel	Only the columns the admin marked 'Apply filter' get a line in the filter panel (list, text, number range, date range with presets, yes/no). Applied filters show as chips.	Everyone
4	KPI cards and charts from the definition	13 chart types, readable axes, funnel, numbers written on the marks, smooth motion. Cards and charts follow the search and filters.	Everyone
5	Create and manage views	A '+' tab to save the screen as a view. Menu: save, update, save as new, remove.	Everyone
6	Views kept on the server	Views follow a person to any device. Older views saved in the browser move up to the server the first time the source is opened. An admin can also share a view with everybody.	Everyone; sharing needs the permission
7	Resize and move columns	Drag a header edge to change the width, drag a header to move it, or use the keyboard. 'Reset column layout' undoes it. Saved inside views.	Everyone
8	Files inside rows (APIs and any source)	A column of file links, Connectum file ids or small pictures shows file chips. Click opens a preview (picture or PDF) or Open / Download.	Everyone
9	No more browser pop-up boxes	Every native confirm / alert / prompt in the app (about 30 places) was replaced by one themed dialog, and a test stops them coming back.	Everyone
10	Fixes found while testing	Phone layout broken by an old saved sidebar width; dialog focus; chart axis, funnel and Configure tab clipping; scroll gaps; a name clash in the CSS.	Everyone

2. How the flow works

Read the picture from the top left. The same path is used for every kind of source, so a new source needs no new screen code.



The six steps

- **Open.** `#view name` opens a source straight away. `#view` alone lists what the person may open. The list is built from calls the app already makes (`ai.enterprise`, `datasource.list`, `api.list`, `isource.list`), so nobody sees a source they could not already see.
- **Two reads together.** The rows come from the source (a live call; a structure's records come from `submissions.list`). The definition comes from `smartview.attrs`. They run in parallel so the screen is not slower.
- **Build the table.** `buildTable` turns the answer into columns and typed cells. A column's type is read from its values: number, date, yes/no, list, JSON, **file**, or text. `applyAttrs` then applies the definition: only the listed columns, their captions, hidden or key columns, links.
- **Work out what to show.** Search, filter, sort and group are each worked out once per change (memoised), never on every scroll.
- **Draw.** KPI cards and charts come only from the definition. The table draws only the rows in view (a window of 48 px rows with 24 rows of margin), so 2,000 rows keep about 30 in the page.
- **Views.** A view is the saved set-up of the screen. It is loaded from the server when the source opens and saved to the server on every change.

> **The most important rule.** A source nobody set up is a **plain table**: no KPI cards, no charts, nothing worked out for it. The menu says 'No KPI' and 'No charts'. Only the inbox lists (Tasks, Approvals, Reminders) work out their own cards. Developers must not add built-in cards for a source.

3. The pieces (where to look)

Screen code, all under `connectum/chat-app/features/smartView/`

File	Its one job
SmartView.jsx	The screen: table, header, menu, view tabs, dialogs. Holds the state (search, filters, sort, group, hidden, order, widths).
SmartViewModal.jsx	The full-screen window with the list of sources; also the small card inside the chat.
smartViewEngine.js	Pure logic, no React: build the table, infer types, search, filter, sort, group, cards, downloads (CSV, TSV, Excel data).
smartViewSources.js	Reads: sources, rows, definition, views (loadViews, saveMyViews, publishView, unpublishView). Talks to the server over the socket.
smartViewAttrs.js	Pure: the definition rules, limits, applyAttrs, defined KPI cards and charts, the type lists (COLUMN_TYPES).
SvConfigure.jsx	The Configure dialog: four grids (Metadata, Hyperlinks, KPIs, Charts), Fill grid, checks before saving.
SvFilterBar.jsx, svFilters.js	The filter panel and the pure code that turns it into conditions.
SvFieldPicker.jsx	The Columns and Group pickers.
SvInsights.jsx, SvChart.jsx, svChartGeometry.js, CountUp.jsx	KPI strip, chart row, the chart drawing (measured text, funnel, axes) and number motion.
svViews.js	Pure: what a view holds, de-duplicating, the Default View (id __default).
svLayout.js	Pure: column order, widths (64 to 800 px), the grid template.
svFiles.js, SvFiles.jsx	Pure: what is a file and what may be opened. The file chips and the preview dialog.
SvDialog.jsx, AppDialog.jsx	The one themed dialog used instead of browser boxes (SvDialog inside Smart View; confirmDialog / alertDialog for the rest of the app).
smartView.css, svDefined.css	Styles. Reuse existing classes before adding new ones.

Server code, under `connectum/connectum-backend/src/`

File	Its one job
forms/sd_smartview.erl	Stores and checks the definition per source. Allowed types: Character, Numeric, Date, Boolean, File.
forms/sd_svviews.erl	Stores and checks views: a person's own and the shared ones. Only a fixed list of keys is kept, each bounded in size.
core/sd_cmds.erl	Actions anyone signed in may call: smartview.attrs, smartview.views, smartview.views.save.
core/sd_cmds_admin.erl	Admin actions: admin.smartview.save, .delete, .view.publish, .view.unpublish. Deleting a data source also removes its definition and shared views.
people/sd_perm.erl	The permission smartview.configure ('Set up Smart views'). Any action starting with admin.smartview. needs it. Administrators always have it.

4. The definition, the storage and the actions

What a definition looks like

One definition per source, found by kind and name (lower case). This is what Configure saves:

```
{ "kind": "list", "name": "demo_sales_order_data",
  "metadata": [
    { "field": "docid", "caption": "SO No", "type": "Character",
      "key": true, "hide": false, "filter": true } ],
  "hyperlinks": [
    { "field": "docid", "type": "tstruct", "target": "Sales Order", "inline": false,
      "mappings": [ { "from": "docid", "to": "docid" } ] } ],
  "kpis": [
    { "caption": "Total Sales", "agg": "sum", "column": "gross_amount" } ],
  "charts": [
    { "caption": "Channel Sales", "chart": "pie", "groupBy": "channel",
      "agg": "sum", "column": "net_amount" } ] }
```

Where things are kept (Redis, one set of keys per organisation)

Key (hash)	Field	Holds
sd:svattrs	<kind>:<name>	The definition of one source.
sd:svviews	<username>|<kind>:<name>	One person's own views of one source.
sd:svshared	<kind>:<name>	The views an admin shared for everybody.

Actions (all over the socket: `/sd action {json}`)

Action	Who	What it does
smartview.attrs {kind, name}	anyone signed in	Returns the definition (or null) and whether this person can configure. Holds names of columns, never data.
admin.smartview.save {kind, name, metadata, hyperlinks, kpis, charts}	smartview.configure	Replaces the definition. The server checks every line and says which line is wrong.
admin.smartview.delete {kind, name}	smartview.configure	Removes the definition: the source goes back to a plain table.
smartview.views {kind, name}	anyone signed in	Returns { mine, shared, canPublish }.
smartview.views.save {kind, name, views}	anyone (own views only)	Replaces the person's own list for the source.
admin.smartview.view.publish / .unpublish	smartview.configure	Shares a view with everybody / stops sharing it.

Limits that keep the data safe

What	Limit
Definition	200 fields, 50 links, 30 KPI cards, 30 charts, 10 mappings per link; every name 1 to 120 characters, no control characters
Views	20 named views per person and source, plus the Default View; 20 shared views per source; one view at most 16 KB; at most 30 filters, 5 group columns, 300 hidden / order / width entries
Column width	64 to 800 px on screen (the server accepts 48 to 1,200 and drops anything else)
Files in a cell	20 files; a picture written into a value at most 2,000,000 characters
Types	Exactly Character, Numeric, Date, Boolean or File (case matters)

5. The three newest features in detail

5.1 Views kept on the server

- A view holds: search, filters, sort, group, hidden columns, **column order, column widths**, and whether KPI cards and charts show.
- Opening a source loads the person's own views and the shared ones. The **Default View** (id `__default`) has no tab; it is what the source opens on.
- Menu on your own view: Save / Update current view, Save as new view, Remove view. On a **shared** view: Save as my own view; for people with the permission also Update shared view and Stop sharing. Remove always asks first.
- **Migration:** the first time a source opens and the server has none, views the older version kept in the browser (local storage) are sent up and the local copy is removed.
- Sources that cannot be configured (the inbox lists) keep their views in the browser, as before.

5.2 Resize and move columns

- **Width:** drag the grip on a header's right edge. With the grip focused, Left / Right arrows change it by 16 px. A double click gives the column its own width back.
- **Order:** drag a header onto another header, or press Alt + Left / Right on a focused header.
- **Reset column layout** appears in the menu only once something was moved or resized.
- The grid template is built in `svLayout.js` so the header and every row always line up.

5.3 Files inside the rows

No set-up is needed. A column is a file column when **every** filled value is a file; Configure can also force the type to File.

The value is...	Result
https://x.test/files/Invoice.pdf	File chip 'Invoice.pdf' (a link ending in a known file type)
{ fileId: '(24 hex characters)', name: 'Scan.png' } or /api/sd/files/(that id)	A Connectum file, fetched with the person's sign-in
{ url, name, mime, size }	File with its own name and type (a declared file needs no extension)
...	A small picture written into the value
a list, JSON text, or links split by a new line, ; or |	Several chips: one name and +N
javascript:..., data:text/html, //host, blob:, file:	Never a file: stays plain text
https://example.com/products/42	Not a file (no file type): stays plain text

The preview draws pictures and PDFs inside a themed dialog (Previous / Next, arrow keys, Escape). Other files offer Open in a new tab or Download. Plain http links and SVG pictures are never drawn inline. Because the server sends every Connectum file as a download with no type, the preview reads the **first bytes** of the file to know what it is (PNG, JPEG, GIF, WebP, PDF) and never trusts the name.

6. What was tested, and what was not

Checked again on 11 Nov 2026 while writing this document

Check	Result
Frontend tests for Smart View (attributes, filters, layout and views, files, no native dialogs)	55 of 55 pass
Lint on the Smart View and services folders	No errors; 2 warnings left that are not from this work (one old unused parameter, one false alarm on Date.now inside a click handler)
Backend suite connectum_smartview_test (isolated backend + Redis in Docker)	85 of 85 passed when last run (82 before the File type, plus 3 new). The backend code has not changed since.
Docker stack (web, backend, Redis) with the demo organisation	Running; backend healthy; demo data and definitions present

Checked by hand in a real Chrome, on the local Docker stack

- Column resize by mouse (220 to 329 px for a 100 px drag), by keyboard (+16 px a press), the 64 px floor, double-click reset; header and rows stay aligned.
- Column move by keyboard; by mouse using drag events (the test tool cannot start a native drag).
- A saved view comes back after a full page reload and after Docker restarts, with order and widths; it is in Redis and not in the browser.
- Sharing a view, the changed menu on a shared view, Stop sharing (confirm, then gone from the server), removing a view (Cancel keeps it, Remove deletes it), moving an old browser view up to the server.
- Light and dark theme, and a phone-width (390 px) frame. This found the phone layout bug, which is fixed.
- An API with 8 rows: file chips for own files, a same-site picture, a plain http link, a picture in the value, an object with name and type, a '+3' cell; a 'javascript:' value stayed plain text.
- Earlier in the work (Configure, filters, charts, hyperlinks, themed dialogs) were checked in the browser and covered by the tests above.

Not verified yet, so please do not assume it works

1. Clicking a file chip and seeing the picture or PDF drawn in the preview was **not** confirmed in Chrome. The detection, the chips and the byte check are tested; the preview dialog itself was not seen working.
2. The larger touch-sized resize grip was not tested (the test browser does not report a touch device).
3. The full frontend suite and the whole backend regression were not run after the last changes (only the Smart View tests).
4. Nothing has run in the CI pipeline: the work is not committed, pushed or merged.

7. Run it and try it

From `connectum/`:

What	Command or place
Frontend tests	node tests/frontend/run.mjs tests/frontend/smartViewAttrs.test.mjs tests/frontend/smartViewFiles.test.mjs ... (or npm test for everything)
Lint	npx oxlint chat-app/features/smartView
Backend Smart View suite	node test/run_all.mjs --exact connectum_smartview_test (from connectum-backend; needs Erlang and Redis, or run it in Docker)
Local stack	docker compose -f deploy/local/docker-compose.yml up -d (web at http://localhost:5173, backend at 8080). Data sources need CONNECTUM_DATA_SOURCES=1.
Demo organisation	Desktop/Connectum_SmartView_Demo/README_Demo_Data.md: sign in with smartviewdemo@test.co, use 'Fill test code' (no password), choose SmartView Demo Co (root admin smartviewadmin).
Open the Smart View	Type #view Sales Order Data in the composer (press Enter twice if the name auto-completes), then Expand.

Two rules for the local web server. (1) Do not run the full frontend suite while the web container is serving the repo: it rewrites public/vendor and the app loses its fonts and libraries. (2) The first load after a restart takes 30 to 60 seconds; the dev server can also stall on this Windows set-up, and restarting the web container fixes it.

Rules this work followed (from CLAUDE.md)

- Production-quality code, no duplication, comments that say why, hostile input handled, docs updated in the same change.
- UI follows UI_DESIGN.md and the liquid-glass guide: reuse existing components, light and dark, phone width, keyboard and Escape, plain-words copy.
- No library from a CDN; nothing was added to the dependencies for this work.
- No push, no pull request, no merge into dev or main; STATUS.md and generated files untouched; feat/orc-agent untouched.

8. How to extend it, and what is still open

Common tasks

I want to...	Do this
Add a column type	Add it to COLUMN_TYPES, TYPE_OF and DECLARED_OF in smartViewAttrs.js; to the TYPES list in sd_smartview.erl (and its error message); teach inferType and formatCell in smartViewEngine.js; add OPS for filters; add tests on both sides.
Add a chart type	Add it to CHART_TYPES in smartViewAttrs.js and draw it in SvChart.jsx; keep the list the server accepts in step.
Add something a view remembers	Add the key in svViews.js (captureView / viewState) and in the whitelist in sd_svviews.erl with a size limit; old views must still open.
Support a new source kind	Add it in smartViewSources.js (list and load) and CONFIGURABLE_KINDS; the rest of the flow does not change.
Show a new kind of file	Extend KINDS in svFiles.js. Draw it inline only if it is safe (never SVG or HTML) and add the byte check in sniffKind.

Not built yet (decided with the owner, waiting)

- **Enable Edit, Enable bulk save, Enable new**: edit rows in the grid and create a record from the view. Agreed scope: structures and Connectum data sources made from one structure; in-grid cells with Save and Bulk save; complex fields open the full form. Needs a server change first: today `update_submission` replaces the whole values map, so a partial update with a conflict check is required.
- **Upload from Excel into a source**: check every row first, import only if all rows are good, report each bad row with row number, column and reason.
- **Server-side paging** beyond the first 500 rows of a data source.

Things to be careful about

- Do not trust a file's name for anything safety-related. For a Connectum file the preview decides from the first bytes. A web link is drawn inline only if it is https and ends in a picture or PDF type (never SVG); only http(s), same-site paths, small pictures and Connectum files are ever opened at all.
- A definition that names a column the source no longer has is skipped quietly, so a changed source never breaks the screen. Do not make it an error.
- Key names in Redis are lower case and per organisation. A view's id starting with 's' and a number means shared; '__default' is the Default View.
- The older Word report on the Desktop (Smart_View_Architecture_and_Change_Report.docx) does not cover views on the server, column layout or files; this PDF replaces it as the current reference.